

CS633 Assignment 1

Group 35

Darshana Baruah
Manas Dhaketa
Jatin Jangra
Amitesh Patra
Om Mallick

February 14, 2026

1 Code Description

This program implements an iterative point-to-point data exchange between MPI processes separated by fixed distances D_1 and D_2 using blocking `MPI_Send` and `MPI_Recv` only.

1.1 Process Roles

Any process with rank r may simultaneously act as:

- Sender to $r + D_1$
- Sender to $r + D_2$
- Receiver from $r - D_1$
- Receiver from $r - D_2$

For all valid processes D_1 and D_2 distance away. The communication occurs for T iterations.

1.2 Sender Validity Conditions

1.2.1 Distance Constraints

For rank r in a communicator world of size P :

D1 is a valid sender if $r + D_1 \leq P - 1$

D2 is a Valid sender if $r + D_2 \leq P - 1$

1.2.2 Boundary Cases

If $r + D_2 > P - 1$, the process sends to D_1 only.

If $r + D_1 > P - 1$, the process does not send to any rank.

1.3 Initialization

Each sender initializes an array of doubles of size M as:

```
 srand(seed);  
 for (int i = 0; i < M; i++)  
     data_D1[i] = (double)rand() * (rank + 1) / 10000.0;  
  
 srand(seed);  
 for (int i = 0; i < M; i++)  
     data_D2[i] = (double)rand() * (rank + 1) / 10000.0;
```

The same seed is used for both D1 and D2 sends initialization.

1.4 Per-Iteration Communication Flow

Each iteration consists of the following sequence:

1. Sender r transmits buffer(s) to valid $r + D_1$ and/or $r + D_2$ ranks.
2. Each receiver performs its respective computations.
3. Receiver sends computed array back to the sender.
4. Sender updates next-iteration buffers for each receiver.

1.4.1 Receiver Computation

$$D_1 : x^2$$
$$D_2 : \log(x)$$

We maintained separate buffers for D1 and D2 paths.

1.5 Buffer Update Rules

After receiving results, a sender updates its buffers for the next iteration :

1.5.1 D1 Update

$$recv_buffer_D1[i] = (unsigned\ long\ long)recv_buffer_D1[i] \bmod 100000$$

1.5.2 D2 Update

$$recv_buffer_D2[i] = recv_buffer_D2[i] \times 100000$$

1.6 Deadlock Avoidance

Communication ordering ensures that all forward sends occur before backward replies. As communication direction is strictly increasing rank for forward sends and decreasing rank for replies, no circular wait condition arises.

1.7 Final Reduction

After T iterations:

- Each valid sender computes:
 - $\max(data_D1)$
 - $\max(data_D2)$
- These maxima are aggregated at rank 0 using a hierarchical pairwise reduction scheme in $\log_2(P)$ stages.
- Rank 0 computes the global maxima across valid senders for $D1$ and $D2$.

1.8 Timing

Timing includes:

- All T iterations
- Final maximum communication

Time is measured using `MPI_Wtime()`. The program prints exactly:

$\langle global_max_D1, global_max_D2, total_time \rangle$

1.9 Algorithm

Algorithm 1 Iterative Distance-Based MPI Data Exchange

```

1: Read  $M, D_1, D_2, T, seed$ 
2: Initialize MPI
3: Obtain rank  $r$  and total processes  $P$ 
4: Allocate buffers for  $D_1$  and  $D_2$ 
5: Initialize data:
6: for  $i = 0$  to  $M - 1$  do
7:    $data\_D1[i] \leftarrow \text{rand}() \times (r + 1)/10000$ 
8: end for
9: for  $i = 0$  to  $M - 1$  do
10:   $data\_D2[i] \leftarrow \text{rand}() \times (r + 1)/10000$ 
11: end for
12: Start timer
13: for  $iter = 1$  to  $T$  do
14:   if  $r + D_1 < P$  then
15:     Send buffer to rank  $r + D_1$ 
16:   end if
17:   if  $r - D_1 \geq 0$  then
18:     Receive array from rank  $r - D_1$ 
19:     for  $i = 0$  to  $M - 1$  do
20:        $recv\_buffer\_D1[i] \leftarrow recv\_buffer\_D1[i]^2$ 
21:     end for
22:     Send result back to rank  $r - D_1$ 
23:   end if
24:   if  $r + D_1 < P$  then
25:     Receive returned  $recv\_buffer\_D1$  array
26:     for  $i = 0$  to  $M - 1$  do
27:        $data\_D1[i] \leftarrow (\text{unsigned long long})recv\_buffer\_D1[i] \bmod 100000$ 
28:     end for
29:   end if
30:   if  $r + D_2 < P$  then
31:     Send buffer to rank  $r + D_2$ 
32:   end if
33:   if  $r - D_2 \geq 0$  then
34:     Receive array from rank  $r - D_2$ 
35:     for  $i = 0$  to  $M - 1$  do
36:        $recv\_buffer\_D2[i] \leftarrow \log(recv\_buffer\_D2[i])$ 
37:     end for
38:     Send result back to rank  $r - D_2$ 
39:   end if
40:   if  $r + D_2 < P$  then
41:     Receive returned  $D_2$  array
42:     for  $i = 0$  to  $M - 1$  do
43:        $data\_D2[i] \leftarrow recv\_buffer\_D2[i] \times 100000$ 
44:     end for
45:   end if
46: end for
47: Compute local maxima of  $data\_D1$  and  $data\_D2$ 
48: Send local maxima to rank 0
49: Rank 0 computes global maxima
50: Stop timer
51: Rank 0 prints:  $\langle global\_max\_D1, global\_max\_D1, total\_time \rangle$ 
52: Free all variables
53: Finalize MPI

```

2 Compilation and Execution

2.1 Compilation

```
mpicc -o execute src.c -lm
```

NOTE: run command ‘mpirun’ before compilation

2.2 Execution

```
mpirun -np <P> -ppn 16 ./execute <M> <D1> <D2> <T> <SEED>
```

2.3 Configurations Tested

- $P = 8, 16, 32$
- $M = 262144, 1048576$
- $D_1 = 2, D_2 = 4$
- $T = 10$
- $seed = 1000$
- Each configuration executed 5 times

3 Results

The program was executed five times for each configuration.

3.1 Runtime Summary

Table 1: Average runtime (seconds) over five executions

Processes (P)	M	Avg Time (s)
8	262144	0.0829
8	1048576	0.3459
16	262144	0.1353
16	1048576	0.5594
32	262144	0.4175
32	1048576	1.1082

3.2 Final Global Maxima

The final maxima obtained after T iterations are summarized for each configuration below. Values shown correspond to the average values over five executions.

Table 2: Final maximum values per configuration

P	M	maximumD1	maximumD2
8	262144	99936.000000	1416360.081579
8	1048576	99936.000000	1416360.081579
16	262144	99936.000000	1416360.081584
16	1048576	99936.000000	1416360.081584
32	262144	99936.000000	1416360.081587
32	1048576	99936.000000	1416360.081587

3.3 Performance Visualization

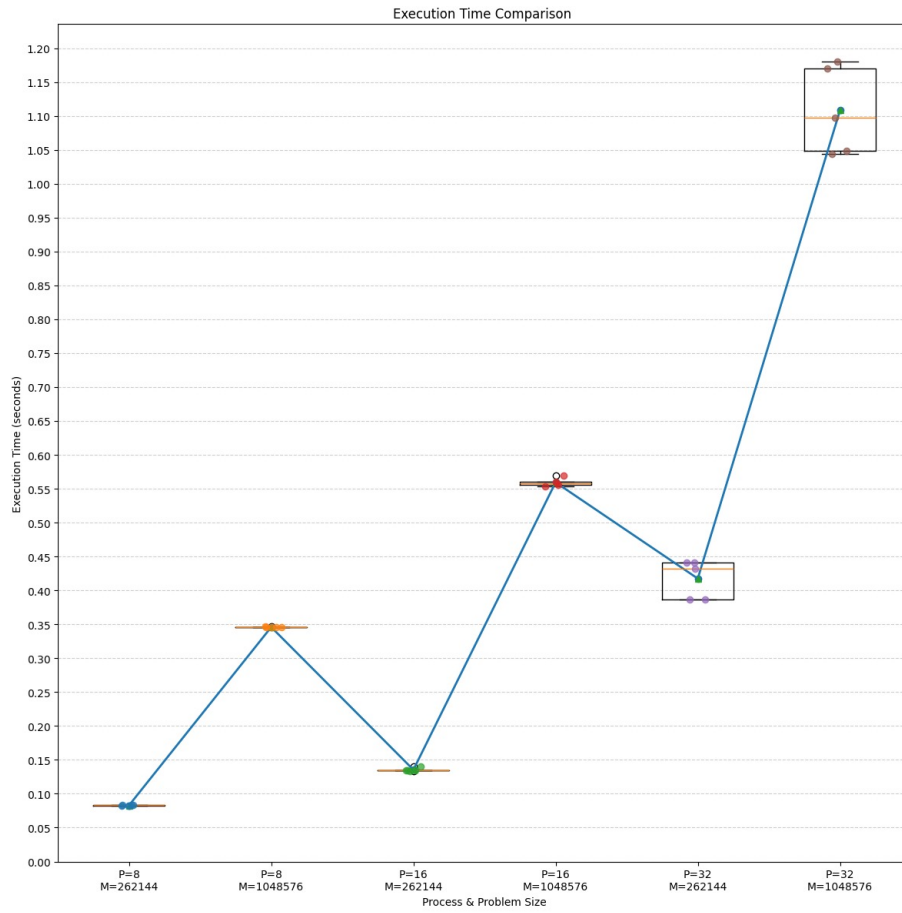


Figure 1: Execution time (seconds) versus number of processes and data size. Each box represents five runs for a fixed number of processes and data size.

4 Observations

4.1 Deterministic Behavior

The value of *global_max_D1* remains identical across all process counts and data sizes for all runs. The value of *global_max_D2* shows negligible variation at the sixth decimal place as P increases. This is attributed to floating-point rounding behavior in distributed computation systems and does not reflect logical differences in execution.

4.2 Scaling with Data Size

Increasing M by a factor of four results in approximately fourfold increase in runtime for $P = 8$ and $P = 16$, indicating that the bandwidth dominated execution.

4.3 Scaling with Process Count

Runtime increases as P increases.

For $M = 262144$:

$$0.082 \rightarrow 0.13 \rightarrow 0.41$$

For $M = 1048576$:

$$0.34 \rightarrow 0.55 \rightarrow 1.108$$

The jump from $P = 16$ to $P = 32$ is high due to increased network contention and inter-node communication.

4.4 Variance Analysis

Variance is minimal for $P = 8$ and $P = 16$. Slightly higher variation appears at $P = 32$ due to increased communication overhead.

4.5 Impact of Inter-Node Communication and Process Placement

- For $P = 8$ and $P = 16$, all processes fit within a single node (16 processes per node), so communication is primarily intra-node and benefits from low shared-memory latency.
- At $P = 32$, processes are distributed across multiple nodes, forcing inter-node communication over the network.
- Inter-node communication introduces higher latency, network bandwidth contention, and additional synchronization overhead.
- Process placement on the supercomputer can further influence performance, as non-optimal rank mapping may increase communication distance and message transfer time.

5 Conclusion

The program correctly implements iterative point-to-point MPI communication using blocking primitives. Experimental evaluation shows:

- $\log(P)$ scaling.
- Increased runtime with higher process counts.
- Communication overhead dominates computation.

Since the workload is not partitioned across processes, increasing P increases communication without reducing per-process computation.